

Bài giảng 5: Tối ưu hóa trong thống kê

TS. Tô Đức Khánh

Khoa Toán-Tin Học, Trường Đại Học Khoa Học Tự Nhiên
Đại Học Quốc Gia Tp. HCM

Thống kê Tính Toán - Computational Statistics

Mục lục

1 *Giới thiệu*

2 *Phương pháp Newton*

3 *Hệ phương pháp Newton-like*

1 Giới thiệu

2 Phương pháp Newton

3 Họ phương pháp Newton-like

Bài toán tối ưu

Bài toán tối ưu là bài toán cổ điển và cực kỳ quan trọng:

$$x^* = \arg \max_{x \in \mathbb{R}} f(x),$$

hoặc

$$x^* = \arg \min_{x \in \mathbb{R}} f(x),$$

với $f(x)$ là hàm thực, có thể là phi tuyến hoặc tuyến tính.

Thông thường, ta giả định rằng, hàm $f(x)$ thỏa

- liên tục;
- khả vi.

Khi đó, nghiệm giải tích x^* được xác định bởi giải phương trình:

$$\frac{df(x)}{dx} = 0.$$

Bài toán tối ưu

Tuy nhiên, có rất nhiều trường hợp, nghiệm giải tích là không thể tìm được.

Ví dụ: xét hàm $f(x) = \frac{\log(x)}{1+x}$, ta muốn tìm

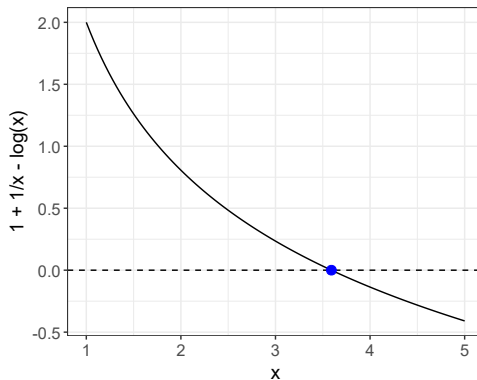
$$x^* = \arg \max_{x \in \mathbb{R}} \frac{\log(x)}{1+x}$$

Nghiệm giải tích x^* là nghiệm của phương trình

$$1 + \frac{1}{x} - \log(x) = 0.$$

$\Rightarrow$ không tìm được nghiệm giải tích, mặc dù nghiệm này tồn tại.

Bài toán tối ưu



⇒ cần một phương pháp giải số để xác định nghiệm.

Bài toán tối ưu hóa đa chiều

Xét không gian $\mathbb{R}^p$ với vector p -chiều, $\mathbf{x} = (x_1, \dots, x_p)^\top$.

Xét hàm thực đa biến, liên tục và khả vi $f(\mathbf{x})$.

Ta quan tâm tới bài toán tối ưu hóa

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in \mathbb{R}^p} f(\mathbf{x}),$$

hoặc

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathbb{R}^p} f(\mathbf{x}).$$

Điểm tối ưu $\mathbf{x}^*$ được xác định bởi giải hệ phương trình đạo hàm

$$\begin{cases} \frac{\partial f(\mathbf{x})}{\partial x_1} &= 0, \\ \vdots &= \vdots, \\ \frac{\partial f(\mathbf{x})}{\partial x_p} &= 0, \end{cases}$$

hay tương đương

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \mathbf{0}.$$

Bài toán tối ưu hóa đa chiều

Đạo hàm cấp hai của $f(\mathbf{x})$ được xác định bởi ma trận Hessian:

$$\mathbf{H}_f(\mathbf{x}) = \frac{\partial^2 f(\mathbf{x})}{\partial \mathbf{x} \partial \mathbf{x}^\top} = \begin{pmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_p} \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_2^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_p} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_p \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_p \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_p^2} \end{pmatrix}.$$

Giải số cho bài toán tối ưu

Do các bài toán tối ưu trong thống kê cũng như máy học có hàm mục tiêu $f(x)$ là phức tạp

⇒ tìm điểm tối ưu thông qua phương pháp giải lặp.

Định nghĩa phương pháp giải lặp

Phương pháp giải lặp (**Iterative methods**) là các kỹ thuật tính toán tạo ra một chuỗi các giải pháp xấp xỉ được cải thiện dần để đạt được mức độ chính xác mong muốn cho các phương trình tuyến tính hoặc phi tuyến.

- Trong mỗi bước lặp thứ t , nghiệm $x^{(t+1)}$ được xác định thông qua một biểu thức của $x^{(t)}$.
- Điểm ban đầu ($t = 0$) là $x^{(0)}$ được dự đoán tùy theo bài toán đang được giải quyết.
- Quá trình lặp sẽ kết thúc khi sai số giữa $x^{(t+1)}$ và $x^{(t)}$ là nhỏ không đáng kể (tức là gần 0).

Giải số cho bài toán tối ưu

Tiêu chuẩn sai số tuyệt đối và tương đối được định nghĩa bởi:

- sai số tuyệt đối

$$D(x^{(t+1)}, x^{(t)}) < \epsilon,$$

- sai số tương đối

$$\frac{D(x^{(t+1)}, x^{(t)})}{D(x^{(t)}, \mathbf{0})} < \epsilon,$$

- sai số tương đối hiệu chỉnh

$$\frac{D(x^{(t+1)}, x^{(t)})}{D(x^{(t)}, \mathbf{0}) + \epsilon} < \epsilon.$$

Giải số cho bài toán tối ưu

Trong không gian $\mathbb{R}^p$, sự sai khác giữa hai vector $\mathbf{u}$ và $\mathbf{v}$, được xác định bởi khoảng cách giữa chúng, $D(\mathbf{u}, \mathbf{v})$. Thông thường,

- khoảng cách Manhattan (L_1 -norm):

$$D(\mathbf{u}, \mathbf{v}) = \sum_{i=1}^p |u_i - v_i|,$$

- khoảng cách Euclidean (L_2 -norm):

$$D(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{i=1}^p (u_i - v_i)^2}.$$

Bài toán tối ưu trong thống kê

Trong thống kê, bài toán tối ưu xuất hiện trong nhiều bài toán khác nhau:

- *likelihood inference*;

ví dụ: xác định tham số chưa biết của hàm mật độ $f(x; \theta)$ của dữ liệu:

$$\theta^* = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log(f(x_i; \theta));$$

- *regression*

ví dụ: phương pháp LASSO (Least Absolute Shrinkage and Selection Operator)

$$\hat{\beta}_{\text{LASSO}} = \arg \max_{\beta \in \mathbb{R}^p} \left\{ \frac{1}{n} \|\mathbf{Y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1 \right\},$$

$\lambda > 0$;

ví dụ: GLM (Generalized linear models) $\mu = g^{-1}(\mathbf{X}\beta)$

$$\hat{\beta} = \arg \max_{\beta \in \mathbb{R}^p} \sum_{i=1}^n \log(f(y_i; \mu(\beta), \sigma^2));$$

Bài toán tối ưu trong thống kê

■ Bayesian decision

ví dụ: cực tiểu hóa hàm rủi ro (risk function)

$$\alpha^* = \arg \min_{\alpha(x)} \sum_{Y=y} L(\alpha(x), y) p(y|x),$$

trong đó, $L(\cdot)$ là hàm tổn thất (loss function).

■ Biomarker evaluation

ví dụ: tìm tổ hợp tuyến tính tối ưu hóa độ chính xác của phân loại 3 nhóm tình trạng bệnh

$$\beta^* = \arg \max_{\beta \in \mathbb{R}^p} \Pr(\mathbf{X}_1\beta < \mathbf{X}_2\beta < \mathbf{X}_3\beta);$$

■ Survival analysis and censored data

ví dụ: mô hình Cox's proportional Hazards

$$\beta^* = \arg \max_{\beta \in \mathbb{R}^p} PL(\beta) = \arg \max_{\beta \in \mathbb{R}^p} \prod_{i:\delta_i=1} \frac{\exp\left(\sum_{j=1}^p X_{ij}\beta_j\right)}{\sum_{i': y_{i'} \geq y_i} \exp\left(\sum_{j=1}^p X_{i'j}\beta_j\right)}.$$

Bài toán tối ưu trong thống kê

Một số trường hợp, ta có thể tìm được nghiệm giải tích.

Ví dụ: cho $X_1, \dots, X_n$ là dữ liệu ngẫu nhiên, độc lập cùng phân phối chuẩn $\mathcal{N}(\mu, \sigma^2)$, với μ, σ^2 chưa biết.

Ước lượng của μ và σ^2 là

$$(\hat{\mu}, \hat{\sigma}^2) = \arg \max_{(\mu, \sigma^2) \in \mathbb{R} \times \mathbb{R}^+} \left\{ -n \log(\sigma^2) - \sum_{i=1}^n \frac{(X_i - \mu)^2}{2\sigma^2} \right\},$$

và công thức tường minh

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i, \quad \hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \hat{\mu})^2.$$

Bài toán tối ưu trong thống kê

Trong hầu hết các trường hợp, ta không tìm được công thức nghiệm.

Ví dụ: cho Y là một mẫu ngẫu nhiên $Y_1, Y_2, \dots, Y_n$, độc lập, từ phân phối Weibull với hàm mật độ xác suất:

$$f(y; \theta, \alpha) = \frac{\alpha}{\theta} \left(\frac{y}{\theta} \right)^{\alpha-1} \exp \left\{ - \left(\frac{y}{\theta} \right)^{\alpha} \right\},$$

với $y > 0$ và $\theta, \alpha > 0$.

Ước lượng của θ và α là

$$\left(\hat{\theta}, \hat{\alpha}\right) = \arg \max_{(\theta, \alpha) \in \mathbb{R}^{+} \times \mathbb{R}^{+}} \left\{ n \log(\alpha) - n \log(\theta) + (\alpha - 1) \sum_{i=1}^n \log\left(\frac{y_i}{\theta}\right) - \sum_{i=1}^n \left(\frac{y_i}{\theta}\right)^{\alpha} \right\}.$$

phương trình đạo hàm

$$-n\alpha/\theta + \alpha\theta^{-1} \sum_{i=1}^n (y_i/\theta)^\alpha = 0$$

$$n/\alpha + \sum_{i=1}^n \log(y_i/\theta) - \sum_{i=1}^n (y_i/\theta)^\alpha \log(y_i/\theta) = 0$$

không tìm được công thức nghiệm giải tích cho α .

1 *Giới thiệu*

2 *Phương pháp Newton*

3 *Họ phương pháp Newton-like*

Phương pháp Newton cho bài toán 1 chiều

Phương pháp Newton hay (*Newton-Raphson*) là phương pháp giải lặp nghiệm dựa trên khai triển Taylor (*Taylor series expansion*).

Nhắc lại rằng, điểm tối ưu x^* của f là nghiệm của phương trình $f'(x) = 0$
 $\Rightarrow f(x^*) = 0$.

Giả sử rằng, ta có một điểm $x^\dagger \in \mathcal{N}_\delta(x^*)$ - lân cận của x^* , khi đó:

$$0 = f'(x^*) = f'(x^\dagger) + (x^* - x^\dagger)f''(x^\dagger) + O((x^* - x^\dagger)^2).$$

Từ đây, ta suy ra

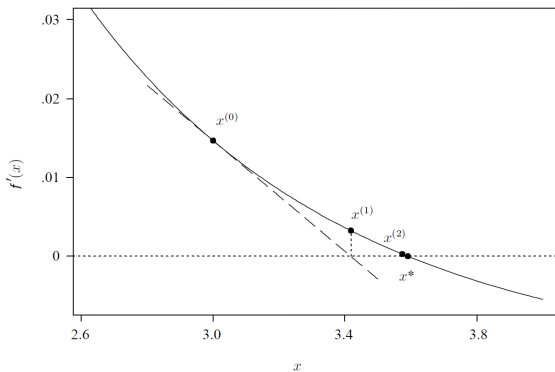
$$x^* = x^\dagger - \frac{f'(x^\dagger)}{f''(x^\dagger)}.$$

Tổng quát hóa, ta có công thức nghiệm lặp

$$x^{(t+1)} = x^{(t)} - \frac{f'(x^{(t)})}{f''(x^{(t)})} = x^{(t)} + h^{(t)},$$

với $t = 0, 1, 2, \dots$ và $h^{(t)}$ được gọi là bước nhảy.

Phương pháp Newton cho bài toán 1 chiều



Các nghiệm lặp $x^{(t)}$ tiến dần về điểm tối ưu x^* .

Phương pháp Newton cho bài toán 1 chiều

Sai số của nghiệm lặp

- sai số tuyệt đối (error)

$$D(x^{(t+1)}, x^{(t)}) = |x^{(t+1)} - x^{(t)}|$$

- sai số tương đối (relative error)

$$D(x^{(t+1)}, x^{(t)}) = \left| \frac{x^{(t+1)} - x^{(t)}}{x^{(t)}} \right|$$

Thuật toán dừng lại tại bước lặp thứ t nếu:

$$D(x^{(t+1)}, x^{(t)}) \leq \epsilon.$$

Thông thường, ta chọn $\epsilon = 10^{-9}$.

Phương pháp Newton cho bài toán 1 chiều

Ví dụ: Xét hàm $f(x) = \frac{\log(x)}{1+x}$.

Ta tính được đạo hàm cấp 1

$$f'(x) = \frac{1 + 1/x - \log(x)}{(1+x)^2}$$

và đạo hàm cấp 2

$$f''(x) = -\frac{3 + 1/x^2 + 4/x - 2\log(x)}{(1+x)^3}.$$

Do đó, ta có bước nhảy

$$h^{(t)} = -\frac{f'(x^{(t)})}{f''(x^{(t)})} = \frac{(1 + 1/x^{(t)} - \log(x^{(t)})) (1 + x^{(t)})}{3 + 1/(x^{(t)})^2 + 4/x^{(t)} - 2\log(x^{(t)})}.$$

Phương pháp Newton cho bài toán 1 chiều

Kết quả của 40 lần lặp

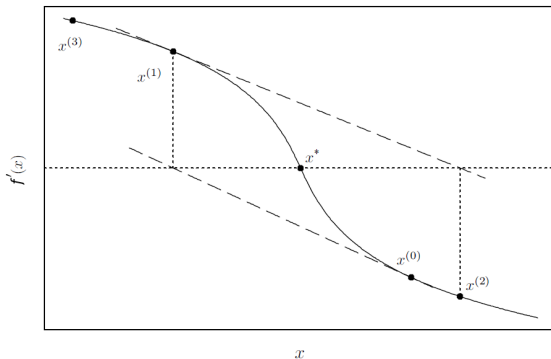
t	$x^{(t)}$	error	relative error
0	3.0000	—	—
1	3.4178	0.4178	0.1393
2	3.5741	0.1563	0.0457
3	3.5910	0.0169	0.0047
4	3.5911	0.0002	4.8827×10^{-5}
5	3.5911	4.4410×10^{-16}	1.2366×10^{-16}
6	3.5911	4.4410×10^{-16}	1.2366×10^{-16}
...	...	...	...
18	3.5911	4.4410×10^{-16}	1.2366×10^{-16}
19	3.5911	4.4410×10^{-16}	1.2366×10^{-16}
...	...	...	...
40	3.5911	4.4410×10^{-16}	1.2366×10^{-16}

⇒ lần lặp thứ 5, nghiệm đã hội tụ.

Sự hội tụ của phương pháp Newton

Phương pháp Newton hội tụ phụ thuộc vào

- hình dạng của hàm f ;
- điểm bắt đầu x^0 .



$\Rightarrow$ bắt đầu từ $x^{(0)}$, phương pháp Newton không hội tụ.

Sự hội tụ của phương pháp Newton

Theorem (Hội tụ trên lân cận)

Xét hàm thực, liên tục và khả vi f . Giả sử rằng các đạo hàm f' , f'' và f''' tồn tại và liên tục, ngoài ra, x^* là nghiệm đơn của f' và $f''(x^*) \neq 0$. Khi đó, tồn tại 1 lân cận $\mathcal{N}_\delta(x^*)$ sao cho phương pháp Newton hội tụ về x^* tại mọi điểm bắt đầu trong lân cận.

Từ đây, ta thu được kết quả sau, Nếu

- 1 $f''(x) \neq 0$ trên $[a, b]$;
- 2 $f'''(x)$ không đổi dấu trên $[a, b]$;
- 3 $f'(a)f'(b) < 0$; và
- 4 $|f'(a)/f''(a)| < b - a$ và $|f'(b)/f''(b)| < b - a$;

thì phương pháp Newton sẽ hội tụ về nghiệm x^* với mọi điểm bắt đầu $x^{(0)} \in [a, b]$.

Thêm vào đó, ta chứng minh được rằng, bậc hội tụ của phương pháp Newton là bậc 2.

Phương pháp Fisher Scoring

Fisher scoring là 1 biến thể của phương pháp Newton khi áp dụng cho likelihood inference.

Nhắc lại rằng, trong likelihood inference, ta muốn xác định tham số θ chưa biết của hàm mật độ $f(x; \theta)$ của dữ liệu:

$$\theta^* = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log(f(x_i; \theta)) \equiv \arg \max_{\theta \in \Theta} \ell(\theta).$$

Do hàm mật độ $f(x; \theta)$ là liên tục, khả vi theo θ , nên ta có

$$\ell'(\theta^*) = \ell'(\theta^\dagger) + (\theta^* - \theta^\dagger) \ell''(\theta^\dagger) + O((\theta^* - \theta^\dagger)^2),$$

với $\theta^\dagger \in \mathcal{N}_\delta(\theta^*)$. Kết hợp với $\ell'(\theta^*) = 0$, ta thu được

$$\theta^* = \theta^\dagger + \ell'(\theta^\dagger) (-\ell''(\theta^\dagger))^{-1}.$$

Phương pháp Fisher Scoring

Và do đó, công thức lặp nghiệm

$$\theta^{(t+1)} = \theta^{(t)} + \ell'(\theta^{(t)}) \left(-\ell''(\theta^{(t)}) \right)^{-1}.$$

Trong likelihood inference, $\ell''(\theta^{(t)})$ được định nghĩa là $\mathcal{J}(\theta)$ - observed information matrix,

$$\theta^{(t+1)} = \theta^{(t)} + \ell'(\theta^{(t)}) \left(-\mathcal{J}^{-1}(\theta^{(t)}) \right).$$

Trong một số trường hợp, $\mathcal{J}(\theta)$ có thể:

- không được định nghĩa tốt;
- có công thức phức tạp.

Khi đó, ta thay thế $\mathcal{J}(\theta)$ bằng ma trận thông tin Fisher (*Fisher information matrix*)

$$\mathcal{I}(\theta) = \mathbb{E}_X \left(-\mathcal{J}(\theta) \right).$$

Công thức lặp nghiệm sẽ là

$$\theta^{(t+1)} = \theta^{(t)} + \ell'(\theta^{(t)}) \mathcal{I}^{-1}(\theta^{(t)}).$$

⇒ phương pháp Fisher scoring.

Phương pháp Newton và Fisher scoring trong đa chiều

Trong bài toàn tối ưu nhiều chiều, phương pháp Newton và Fisher scoring có thể mở rộng dựa trên khai triển Taylor trên $\mathbb{R}^p$:

$$\mathbf{0} = \frac{\partial f(\mathbf{x}^*)}{\partial \mathbf{x}} = \frac{\partial f(\mathbf{x}^\dagger)}{\partial \mathbf{x}} + \mathbf{H}_f(\mathbf{x}^\dagger)(\mathbf{x}^* - \mathbf{x}^\dagger) + O(D(\mathbf{x}^*, \mathbf{x}^\dagger)^2).$$

Từ đây, ta suy ra

$$\mathbf{x}^* = \mathbf{x}^\dagger - \mathbf{H}_f^{-1}(\mathbf{x}^\dagger) \frac{\partial f(\mathbf{x}^\dagger)}{\partial \mathbf{x}}.$$

Do đó, công thức nghiệm lặp cho phương pháp Newton

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \mathbf{H}_f^{-1}(\mathbf{x}^{(t)}) \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}}.$$

Phương pháp Newton và Fisher scoring trong đa chiều

Đối với likelihood inference trong trường hợp vector tham số $\theta \in \mathbb{R}^p$, ta có:

- hàm mục tiêu (cần cực đại hóa) - hàm log-likelihood

$$\ell(\theta) = \sum_{i=1}^n \log(f(x_i; \theta)),$$

trong đó, $f(x; \theta)$ là hàm mật độ xác suất của dữ liệu;

- đạo hàm cấp 1 của hàm mục tiêu

$$\mathbf{U}(\theta) = \frac{\partial \ell(\theta)}{\partial \theta},$$

- ma trận thông tin Fisher (Fisher information matrix)

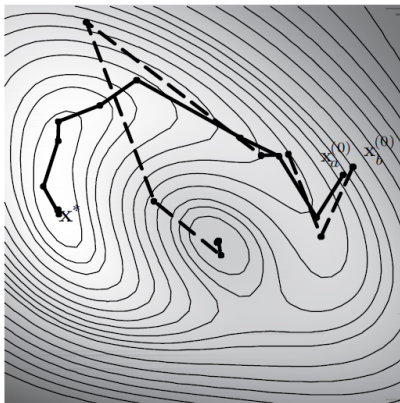
$$\mathcal{I}(\theta) = \mathbb{E}_{\mathbf{X}} \{-\mathbf{H}_{\ell}(\theta)\}$$

Công thức nghiệm lặp cho phương pháp Fisher scoring

$$\theta^{(t+1)} = \theta^{(t)} + \mathcal{I}^{-1}(\theta^{(t)})\mathbf{U}(\theta^{(t)}).$$

Phương pháp Newton và Fisher scoring trong đa chiều

Đối với một số hàm phức tạp (có cực trị địa phương), phương pháp Newton (hoặc Fisher scoring) có thể cho kết quả khác nhau với các điểm bắt đầu khác nhau.



Điểm $x_a^{(0)}$ cho kết quả tối ưu toàn cục; $x_b^{(0)}$ cho kết quả tối ưu đại phương.

Ví dụ: phân phối Weibull

Ta áp dụng mô hình phân phối Weibull để mô hình hóa dữ liệu về thời gian hỏng của lò xo trong thí nghiệm với mức ứng suất 950 N/mm²:

225, 171, 198, 189, 189, 135, 162, 135, 117, 162

Nhắc lại, hàm mật độ của phân phối Weibull:

$$f(y; \theta, \alpha) = \frac{\alpha}{\theta} \left(\frac{y}{\theta}\right)^{\alpha-1} \exp \left\{ - \left(\frac{y}{\theta}\right)^{\alpha} \right\},$$

với $y > 0$ và $\theta, \alpha > 0$.

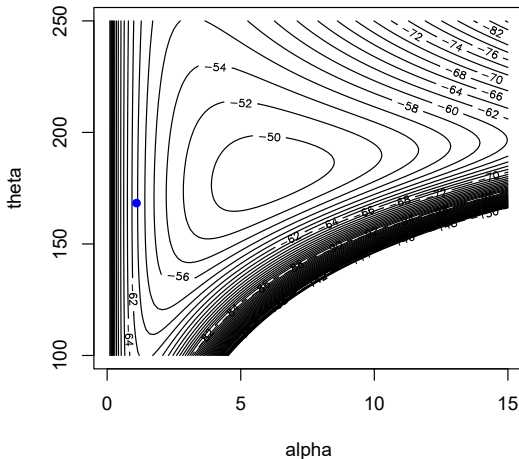
Hàm log-likelihood là

$$\ell(\theta, \alpha) = n \log(\alpha) - n \log(\theta) + (\alpha - 1) \sum_{i=1}^n \log \left(\frac{y_i}{\theta} \right) - \sum_{i=1}^n \left(\frac{y_i}{\theta} \right)^{\alpha}.$$

Do $\theta, \alpha > 0$, nên để đảm bảo nghiệm lặp thỏa tính chất, ta thực hiện phép đổi biến $\psi = (\log(\theta), \log(\alpha))$, sau đó với phép biến đổi ngược, $\exp(\cdot)$, ta thu được kết quả $\hat{\theta}, \hat{\alpha} > 0$.

Ví dụ: phân phối Weibull

Thực hiện giải lặp, với hai điểm bắt đầu $\theta^{(0)} = 168.3000$, $\alpha^{(0)} = 1.1000$, và sai số chặn là 10^{-9} .



Ví dụ: phân phối Weibull

Bảng tổng hợp kết quả

Lần lặp t	$\theta^{(t)}$	$\alpha^{(t)}$	relative error
0	168.3000	1.1000	—
1	172.1154	2.1111	6.5226×10^{-1}
2	175.1454	3.6845	5.5723×10^{-1}
3	180.4374	5.2457	3.5452×10^{-1}
4	181.1004	5.9082	1.1899×10^{-1}
5	181.4075	5.9764	1.1604×10^{-3}
6	181.4056	5.9769	8.1710×10^{-5}
7	181.4056	5.9769	3.6697×10^{-9}
8	181.4056	5.9769	2.2204×10^{-16}

$$\Rightarrow \hat{\theta} = 181.4056, \hat{\alpha} = 5.9769.$$

Phương pháp IRLS

Iteratively Reweighted Least Squares (IRLS) là một biến thể của phương pháp Fisher scoring trong bài toán ước lượng mô hình hồi quy tuyến tính tổng quát (*Generalized Linear Models - GLM*).

Giới thiệu GLM

GLM là sự tổng quát mô hình tuyến tính để tương thích với các dạng dữ liệu:

- biến phản hồi có dạng: nhị thức, số đếm, dạng định tính;
- biến phản hồi có phân phối bất đối xứng: lệch phải, có nhiều giá trị lớn (extreme values);
- phương sai của biến phản hồi trong mô hình thay đổi theo giá trị trung bình.

Gọi $\mu_i = \mathbb{E}(Y_i | X_{1i} = x_{1i}, \dots, X_{pi} = x_{pi})$ của biến phản hồi tại p giá trị biến giải thích $(x_{1i}, \dots, x_{pi})$, liên hệ nó với *linear predictor*,

$$g(\mu_i) = \beta_0 + \beta_1 X_{1i} + \dots + \beta_p X_{pi} \equiv \eta_i,$$

g là hàm liên kết (*link function*). Hàm $g(\cdot)$ là hàm đơn điệu, 1-1, khả vi, nên

$$\mu_i = g^{-1}(\beta_0 + \beta_1 X_{1i} + \dots + \beta_p X_{pi}) \equiv g^{-1}(\eta_i).$$

Phương pháp IRLS

Hệ số $\beta = (\beta_0, \beta_1, \dots, \beta_p)^\top$ của GLM được xác định bởi cực đại hóa

$$\hat{\beta} = \arg \max_{\beta \in \mathbb{R}^p} \sum_{i=1}^n \log(f(y_i; \mu(\beta), a(\phi))) = \arg \max_{\beta \in \mathbb{R}^p} \ell(\beta, a(\phi));$$

trong đó, $f(y_i; \cdot)$ là phân phối của biến phản hồi, và thường thuộc vào họ phân phối mũ phân tán (**exponential-dispersion family**):

$$f_{Y_i}(y_i | \theta_i, \phi) = \exp \left(\frac{y_i \theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi) \right),$$

trong đó,

- θ_i được gọi là tham số tự nhiên (*natural parameter*);
- $\phi > 0$ được gọi là tham số phân tán (*dispersion parameter*).

Phương pháp IRLS

Đặt

- $\mathbf{Y} = (Y_1, \dots, Y_n)^\top,$

- $\mathbf{X}_i = (1, X_{1i}, \dots, X_{pi})^\top$,

$$\mathbf{X} = \begin{pmatrix} 1 & X_{11} & X_{21} & \dots & X_{p1} \\ 1 & X_{12} & X_{22} & \dots & X_{p2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & X_{1n} & X_{2n} & \dots & X_{pn} \end{pmatrix},$$

- $\beta = (\beta_0, \beta_1, \dots, \beta_p)^\top$,

- $\eta_i \equiv \eta_i(\beta) = \mathbf{x}_i^\top \beta,$

- $\eta \equiv \eta(\beta) = \mathbf{X}\beta$.

Phương pháp IRLS

Do $Y_i \sim \text{EDM}(b(\theta_i), a(\phi)) \Rightarrow \mathbb{E}(Y_i) = \mu_i = b'(\theta_i)$.

Mặt khác, ta cũng có

- $\mu_i = g^{-1}(\eta_i)$;
- $b'(\theta_i) = \mu_i$;

$\Rightarrow$ ta có thể viết θ_i dưới dạng hàm ẩn của β , như sau $\theta_i \equiv \theta_i(\mu_i(\eta_i(\beta)))$.

Từ đây, ta có hàm mật độ xác suất của Y_i là

$$f(Y_i; \beta, a(\phi)) = \exp \left\{ \frac{Y_i \theta_i(\mu_i(\eta_i(\beta))) - b(\theta_i(\mu_i(\eta_i(\beta))))}{a(\phi)} + c(Y_i, \phi) \right\},$$

và do đó, hàm log-likelihood là

$$\ell(\beta, a(\phi)) = \sum_{i=1}^n \ell_i(\beta, a(\phi)) = \sum_{i=1}^n \left\{ \frac{Y_i \theta_i(\mu_i(\eta_i(\beta))) - b(\theta_i(\mu_i(\eta_i(\beta))))}{a(\phi)} + c(Y_i, \phi) \right\}.$$

Bởi vì các hàm $b(\cdot)$ và $g(\cdot)$ là liên tục và khả vi nên ta có thể tính được đạo hàm của chúng, cũng như là hàm ẩn $\theta_i(\mu_i(\eta_i(\beta)))$.

Phương pháp IRLS

Hàm score cho GLM, $\mathbf{U}(\beta)$ được xác định bởi

$$\mathbf{U}(\beta) = \frac{\partial \ell(\beta, \mathbf{a}(\phi))}{\partial \beta} = \sum_{i=1}^n \frac{d\ell_i}{d\theta_i} \frac{d\theta_i}{d\mu_i} \frac{d\mu_i}{d\eta_i} \frac{\partial \eta_i}{\partial \beta}.$$

Áp dụng các quy tắc đạo hàm ẩn và hàm ngược, ta thu được

$$\mathbf{U}(\beta) = \frac{1}{a(\phi)} \sum_{i=1}^n W_i g'(\mu_i) (Y_i - \mu_i) \mathbf{X}_i,$$

trong đó, $W_i = \frac{1}{V(\mu_i) (g'(\mu_i))^2}$ và được gọi là *working weights*.

Phương pháp IRLS

Hàm score $\mathbf{U}(\beta)$ được viết lại dưới dạng ma trận như sau:

$$\mathbf{U}(\beta) = \frac{1}{a(\phi)} \mathbf{X}^\top \mathbf{W} \mathbf{G} (\mathbf{Y} - \boldsymbol{\mu}),$$

trong đó

$$\begin{aligned} \blacksquare \mathbf{W} &= \begin{pmatrix} W_1 & 0 & \dots & 0 \\ 0 & W_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & W_n \end{pmatrix}, \\ \blacksquare \mathbf{G} &= \begin{pmatrix} g'(\mu_1) & 0 & \dots & 0 \\ 0 & g'(\mu_2) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & g'(\mu_n) \end{pmatrix}, \\ \blacksquare \boldsymbol{\mu} &= (\mu_1, \mu_2, \dots, \mu_n)^\top. \end{aligned}$$

Phương pháp IRLS

Với biểu thức tổng quát của $\mathbf{U}(\beta)$, ta tìm được ma trận thông tin quan sát:

$$\begin{aligned}
\mathcal{J}(\beta) &= -\frac{\partial \mathbf{U}(\beta)}{\partial \beta} \\
&= -\frac{1}{a(\phi)} \sum_{i=1}^n \left\{ \left[\frac{\partial}{\partial \beta} (W_i g'(\mu_i)) \right] (Y_i - \mu_i) - W_i g'(\mu_i) \frac{\partial \mu_i}{\partial \beta} \right\} \mathbf{x}_i.
\end{aligned}$$

Nhận xét rằng,

- $\mathbb{E}(Y_i) = \mu_i$;
- $\frac{\partial \mu_i}{\partial \beta} = \frac{d\mu_i}{d\eta_i} \frac{\partial \eta_i}{\partial \beta} = \frac{1}{g'(\mu_i)} \mathbf{x}_i$.

Khi đó, ta tính được ma trận thông tin Fisher

$$\mathcal{I}(\beta) = \mathbb{E}(\mathcal{J}(\beta)) = \frac{1}{a(\phi)} \sum_{i=1}^n W_i \mathbf{x}_i \mathbf{x}_i^\top = \frac{1}{a(\phi)} \mathbf{x}^\top \mathbf{W} \mathbf{x}.$$

Phương pháp IRLS

Áp dụng phương pháp Fisher Scoring:

$$\beta^{(t+1)} = \beta^{(t)} + \mathcal{I}^{-1}(\beta^{(t)}) \mathbf{U}(\beta^{(t)}).$$

Cụ thể

$$\beta^{(t+1)} = \beta^{(t)} + (\mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{G}^{(t)} (\mathbf{Y} - \boldsymbol{\mu}^{(t)}),$$

trong đó, $\mathbf{W}^{(t)}$, $\boldsymbol{\mu}^{(t)}$ và $\mathbf{G}^{(t)}$ lần lượt được tính dựa vào hệ số $\beta^{(t)}$.

Mặt khác, với cách đặt $\boldsymbol{\eta}^{(t)} = \mathbf{X}\beta^{(t)}$ và $\mathbf{z}^{(t)} = \boldsymbol{\eta}^{(t)} + \mathbf{G}^{(t)} (\mathbf{Y} - \boldsymbol{\mu}^{(t)})$, ta có

$$\beta^{(t+1)} = (\mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{z}^{(t)},$$

⇒ công thức lặp được gọi là **lặp bình phương nhỏ nhất với trọng số được tính lại (iteratively re-weighted least square)**, hay viết tắt bởi **IRLS**.

Chú ý: trong biểu thức lặp của $\beta^{(t+1)}$, không có sự xuất hiện của tham số $a(\phi)$, vì vậy ước lượng $\hat{\beta}$ được tính mà không cần thông tin của $a(\phi)$.

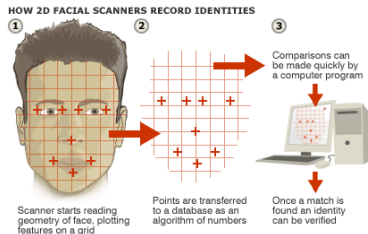
Ma trận phương sai của $\hat{\beta}$ là

$$\text{Var}(\hat{\beta}) = \mathcal{I}^{-1}(\hat{\beta}) = (\mathbf{X}^\top \mathbf{W}(\hat{\beta}) \mathbf{X})^{-1}$$

Ví dụ: Human Face Recognition

Thuật toán nhận diện khuôn mặt người thường tập trung 3 yếu tố

- sự khác biệt về cường độ điểm ảnh ở vùng mắt (eyediff);
- sự khác biệt về cường độ điểm ảnh ở vùng mũi/má (nosecheekdiff);
- sự thay đổi cường độ điểm ảnh so sánh giữa hai hình ảnh của cùng một người (variabilityratio).



Dựa trên các yếu tố này, thuật toán sẽ so sánh 1 bức hình của 1 người với các hình còn lại (có duy nhất 1 hình của cùng 1 người), kết quả trả về:

- khớp (xác định đúng hình của người đang xét) - 1;
- không khớp - 0.

Ví dụ: Human Face Recognition

Để đánh giá sự ảnh hưởng của các yếu tố này tới khả năng nhận diện đúng của thuật toán, một thí nghiệm được tiến hành:

	match	eyediff	nosecheekdiff	variabilityratio
1	1	0.0096827670	0.027914422	1.0862036
2	1	0.0276138930	0.017821209	0.9898404
3	1	0.0153665190	0.025831893	1.0109830
4	1	0.0133714650	0.024047631	1.0127243
5	1	0.0114276410	0.017800852	1.0443961
6	1	0.0686048350	0.050388073	0.9508176
7	1	0.0092405937	0.016042421	1.0067264
8	1	0.0303490000	0.014707760	1.0186794
9	1	0.0310754380	0.069049962	1.0307903
10	0	0.0072111477	0.099909148	0.9289499

Ví dụ: Human Face Recognition

Ta quan tâm tới yếu tố eyediff và muốn đánh giá sự ảnh hưởng của yếu tố này tới xác suất nhận diện đúng $\Pr(\text{match} = 1 | \text{eyediff})$.

Một mô hình được đề xuất là mô hình logistic

$$\Pr(\text{match}_i = 1 | \text{eyediff}_i) = \frac{\exp(\beta_0 + \beta_1 \text{eyediff}_i)}{1 + \exp(\beta_0 + \beta_1 \text{eyediff}_i)},$$

$i = 1, \dots, n$. Đặt $\pi_i = \Pr(\text{match}_i = 1 | \text{eyediff}_i)$ và $\eta_i(\beta) = \beta_0 + \beta_1 \text{eyediff}_i$.

Đối với mô hình logistic

- $\mu_i = \pi_i$;
- hàm liên kết $g(u) = \frac{\exp(u)}{1 + \exp(u)}$;
- $a(\phi) = 1$
- hàm log-likelihood

$$\ell(\beta) = \sum_{i=1}^n \{Y_i \eta_i(\beta) - \log(1 + \exp(\eta_i(\beta)))\}$$

Ví dụ: Human Face Recognition

Ta tìm công thức nghiệm lặp IRLS

$$\beta^{(t+1)} = \beta^{(t)} + (\mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{Y} - \boldsymbol{\pi}^{(t)}),$$

với

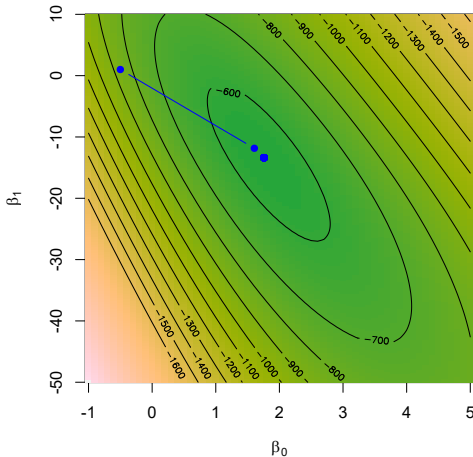
$$\mathbf{Y} = \begin{pmatrix} \text{match}_1 \\ \text{match}_2 \\ \vdots \\ \text{match}_n \end{pmatrix}, \quad \boldsymbol{\pi} = \begin{pmatrix} \pi_1 \\ \pi_2 \\ \vdots \\ \pi_n \end{pmatrix} \quad \mathbf{X} = \begin{pmatrix} 1 & \text{eyediff}_1 \\ 1 & \text{eyediff}_2 \\ \vdots & \vdots \\ 1 & \text{eyediff}_n \end{pmatrix}$$

và

$$\mathbf{W} = \begin{pmatrix} \pi_1(1 - \pi_1) & 0 & \dots & 0 \\ 0 & \pi_2(1 - \pi_2) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \pi_n(1 - \pi_n) \end{pmatrix}$$

Ví dụ: Human Face Recognition

Với điểm bắt đầu $\beta^{(0)} = (\beta_0^{(0)}, \beta_1^{(0)}) = (-0.5, 1)$, quá trình lặp được biểu diễn bởi hình dưới đây:



Ví dụ: Human Face Recognition

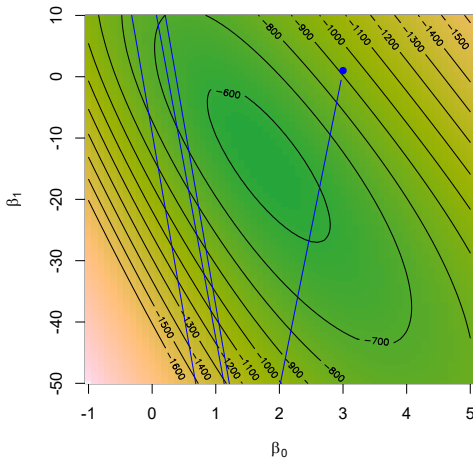
Kết quả của 40 lần lặp

t	$\beta_0^{(t)}$	$\beta_1^{(t)}$	error	relative error
0	-0.5000	1.0000	—	—
1	1.6063	-11.8460	13.0175	11.6432
2	1.7535	-13.3465	1.5078	0.1261
3	1.7587	-13.3400	0.0537	0.0040
4	1.7587	-13.4000	6.7048×10^{-5}	4.9611×10^{-6}
5	1.7587	-13.4000	1.0302×10^{-10}	7.6228×10^{-12}
6	1.7587	-13.4000	3.6621×10^{-15}	2.7096×10^{-16}
⋮	⋮	⋮	⋮	⋮
18	1.7587	-13.4000	1.7763×10^{-15}	1.3144×10^{-16}
19	1.7587	-13.4000	1.7902×10^{-15}	1.3246×10^{-16}
⋮	⋮	⋮	⋮	⋮
40	1.7587	-13.4000	3.5804×10^{-15}	2.6492×10^{-16}

$\Rightarrow \hat{\beta}_0 = 1.7587, \hat{\beta}_1 = -13.4000$, sau 5 lần lặp.

Ví dụ: Human Face Recognition

Tuy nhiên, với điểm bắt đầu $\beta^{(0)} = (\beta_0^{(0)}, \beta_1^{(0)}) = (3, 1)$, IRLS **không** hội tụ.



Phương pháp IRLS: chọn điểm bắt đầu

Nhận thấy rằng, ở lần lặp đầu tiên của IRLS, $\mathbf{W}^{(0)}$ và $\mathbf{z}^{(0)}$ chỉ phụ thuộc vào hệ số $\beta^{(0)}$ thông qua giá trị trung bình $\mu_i^{(0)}$.

Do đó, ta có thể chọn trực tiếp giá trị bắt đầu $\mu_i^{(0)}$ thay vì phải tính.

Mặt khác, mục tiêu của việc sử dụng GLM là để ước lượng $\hat{\mu}_i$ sao cho gần nhất có thể với giá trị quan sát Y_i .

$\Rightarrow$ ta có thể bắt đầu thuật toán bằng cách chọn giá trị bắt đầu $\mu_i^{(0)} = Y_i$.

Chú ý kỹ thuật: khi hàm liên kết của mô hình có dạng logarithm hoặc nghịch đảo $\Rightarrow$ thuật toán không thể khởi động nếu $Y_i = 0$.

$\Rightarrow$ ta cần phải áp dụng một số tinh chỉnh nhỏ, ví dụ:

- $\mu_i^{(0)} = Y_i + 0.1$;
- $\mu_i^{(0)} = (mY_i + 0.5)/(m + 1)$, thay vì đặt $\mu_i^{(0)} = 0$ hoặc 1, đối với trường hợp của phân phối nhị thức.

Cách chọn điểm bắt đầu như thế này giúp thuật toán IRLS hội tụ nhanh, và nó cũng được áp dụng trong các phần mềm thống kê.

3 Họ phương pháp Newton-like

Động lực

Trong phần trước, với phương pháp Newton và Fisher Scoring, công thức nghiệm lặp luôn có dạng chung

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \mathbf{M}^{-1}(\mathbf{x}^{(t)}) \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}},$$

trong đó, ma trận $\mathbf{M}(\mathbf{x}^{(t)})$ có thể là

- $\mathbf{M}(\mathbf{x}^{(t)}) = \mathbf{H}_f(\mathbf{x}^{(t)})$ – phương pháp Newton;
- $\mathbf{M}(\mathbf{x}^{(t)}) = \mathcal{I}(\mathbf{x}^{(t)})$ – phương pháp Fisher Scoring.

Có 02 nhược điểm của phương pháp Newton:

- chi phí tính ma trận Hessian có thể lâu;
- ở mỗi lần lặp, các bước nhảy của không đảm bảo rằng giá trị của hàm đi lên (uphill), tức là $f(\mathbf{x}^{(t+1)}) > f(\mathbf{x}^{(t)})$.

⇒ việc thay thế $\mathbf{H}_f(\mathbf{x}^{(t)})$ bằng 1 dạng xấp xỉ $\mathbf{M}(\mathbf{x}^{(t)})$ phù hợp, có thể giúp giải quyết các vấn đề này.

Các thuật toán Ascent

Nhắc lại rằng, trong bài toán tìm cực đại, x^* là điểm cực trị nếu và chỉ nếu $\frac{\partial f(x^*)}{\partial x} = \mathbf{0}$ và $-\mathbf{H}_f(x^*)$ xác định dương.

Đối với phương pháp Newton, tại một lần lặp, $-\mathbf{H}_f(x^{(t)})$ có thể xác định âm.

$\Rightarrow$ ta thay thế $-\mathbf{H}_f(x^{(t)})$ bằng một dạng xấp xỉ $-\mathbf{M}^{(t)}$ chắc chắn xác định dương.

$\Rightarrow$ những thuật toán sử dụng ma trận xấp xỉ như vậy được gọi là **ascent algorithms** và sẽ “ép buộc” nghiệm lặp phải tiến lên vùng cực đại (uphill).

Các thuật toán Ascent

Có nhiều dạng $-\mathbf{M}^{(t)}$ khác nhau sẽ tạo ra thuật toán ascent, với nghiệm lặp

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \left(\mathbf{M}^{(t)}\right)^{-1} \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}}.$$

Tuy nhiên, trong giải lặp, điểm $\mathbf{x}^{(t+1)}$ có thể “bước” quá xa, vượt ra khỏi vùng cực đại.

$\Rightarrow$ ta cần 1 hệ số tỷ lệ để hiệu chỉnh lại bước nhảy không quá lớn. Khi này, công thức nghiệm lặp là

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha^{(t)} \left(\mathbf{M}^{(t)}\right)^{-1} \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}},$$

với $\alpha^{(t)} > 0$ giúp điều khiển sự hội tụ.

Các thuật toán Ascent

Với bất kỳ $\mathbf{x}^{(t)}$ cố định, và ma trận $\mathbf{M}^{(t)}$ xác định âm, khi $\alpha^{(t)} \rightarrow 0$, theo khai triển Taylor bậc 1, ta có biểu diễn:

$$\begin{aligned} f(\mathbf{x}^{(t+1)}) &= f(\mathbf{x}^{(t)}) + \left\{ \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}} \right\}^\top (\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}) \\ &\quad + o\left((\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)})^\top (\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)})\right) \\ &= f(\mathbf{x}^{(t)}) - \left\{ \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}} \right\}^\top \alpha^{(t)} (\mathbf{M}^{(t)})^{-1} \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}} + o(\alpha^{(t)}) \end{aligned}$$

Do đó,

$$f(\mathbf{x}^{(t+1)}) - f(\mathbf{x}^{(t)}) = - \left\{ \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}} \right\}^\top \alpha^{(t)} (\mathbf{M}^{(t)})^{-1} \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}} + o(\alpha^{(t)})$$

Nếu $-\mathbf{M}^{(t)}$ xác định dương, thì với cách chọn $\alpha^{(t)}$ đủ nhỏ, sẽ đảm bảo rằng

$$f(\mathbf{x}^{(t+1)}) - f(\mathbf{x}^{(t)}) > 0.$$

Các thuật toán Ascent

Trong cách các chọn $\mathbf{M}^{(t)}$, cách chọn đơn giản nhất là $\mathbf{M}^{(t)} = -\mathbf{I}$, tức là ma trận đơn vị.

⇒ phương pháp **steepest ascent** (đường lên dốc nhất).

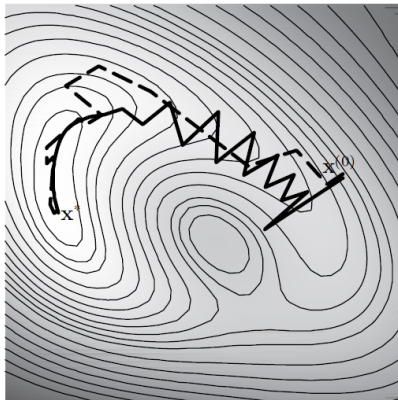
Ngoài ra, khi kết hợp với phương pháp Fisher scoring, ta có thể chọn $-\mathbf{M}^{(t)} = \mathcal{I}(\theta^{(t)})$.

Trong thực hiện thuật toán, tại bước lặp $t + 1$, $\alpha^{(t)}$ có thể được chọn như sau:

- 1 khởi tạo $\alpha^{(t)} = 1$;
- 2 xác định $\mathbf{x}^{(t+1)}$;
- 3 nếu $f(\mathbf{x}^{(t+1)}) < f(\mathbf{x}^{(t)})$ thì tính lại $\mathbf{x}^{(t+1)}$ và $\alpha^{(t)} := \alpha^{(t)}/2$;

lặp lại 1 - 3, tới khi nào $f(\mathbf{x}^{(t+1)}) - f(\mathbf{x}^{(t)}) > 0$.

Các thuật toán Ascent



Đường nét liền màu đen miêu tả đường di chuyển của $x^{(t+1)}$.

Thuật toán steepest ascent hội tụ thành công nhưng khá chậm và không hiệu quả.

Các phương pháp quasi-Newton

Tiếp tục khai thác ý tưởng về công thức nghiệm lặp

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha^{(t)} (\mathbf{M}^{(t)})^{-1} \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}},$$

với $\alpha^{(t)} > 0$.

Thay vì cố định $\mathbf{M}^{(t)} = -\mathbf{I}$ như phương pháp steepest ascent, một ý tưởng là cập nhật $\mathbf{M}^{(t)}$ sau mỗi lần lặp nhưng vẫn giữ được tính chất tiến dần của nghiệm.

Một số ký hiệu

Đặt

- $\mathbf{h}^{(t)} = \mathbf{x}^{(t+1)} - \mathbf{x}^{(t)};$
- $\mathbf{u}^{(t)} = \frac{\partial f(\mathbf{x}^{(t+1)})}{\partial \mathbf{x}} - \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}}.$

Các phương pháp quasi-Newton

Phương pháp symmetric rank-one

Bắt đầu với $\mathbf{M}^{(0)} = -\mathbf{I}$, phương pháp này cập nhật

$$\mathbf{M}^{(t+1)} = \mathbf{M}^{(t)} + c^{(t)} \mathbf{v}^{(t)} (\mathbf{v}^{(t)})^\top,$$

với $t = 0, 1, 2, \dots$; $\mathbf{v}^{(t)} = \mathbf{u}^{(t)} - \mathbf{M}^{(t)} \mathbf{h}^{(t)}$ và $c^{(t)} = 1 / \left[(\mathbf{v}^{(t)})^\top \mathbf{h}^{(t)} \right]$.

Để chắc chắn về sự xác định dương của $-\mathbf{M}^{(t+1)}$, thì $c^{(t)}$ phải được xác định, tức là

$$\left| (\mathbf{v}^{(t)})^\top \mathbf{h}^{(t)} \right| > \epsilon,$$

với $\epsilon > 0$. Nếu điều kiện này không xảy ra thì $\mathbf{M}^{(t+1)}$ sẽ được đặt là $\mathbf{M}^{(t)}$.

Mặt khác,

- nếu $c^{(t)} \leq 0$ và $-\mathbf{M}^{(t)}$ là xác định dương thì $\mathbf{M}^{(t+1)}$ **có thể** xác định dương;
- nếu $c^{(t)} > 0$ thì cần phải quay lại để “giảm” giá trị $c^{(t)}$ về hướng 0, cho tới khi sự xác định dương đạt được.

Các phương pháp quasi-Newton

Phương pháp symmetric rank-two

Bắt đầu với $\mathbf{M}^{(0)} = -\mathbf{I}$, phương pháp này cập nhật

$$\begin{aligned} \mathbf{M}^{(t+1)} = & \mathbf{M}^{(t)} - \frac{\mathbf{M}^{(t)} \mathbf{h}^{(t)} (\mathbf{M}^{(t)} \mathbf{h}^{(t)})^\top}{(\mathbf{h}^{(t)})^\top \mathbf{M}^{(t)} \mathbf{h}^{(t)}} + \frac{\mathbf{u}^{(t)} (\mathbf{u}^{(t)})^\top}{(\mathbf{h}^{(t)})^\top \mathbf{u}^{(t)}} \\ & + \delta^{(t)} \left((\mathbf{h}^{(t)})^\top \mathbf{M}^{(t)} \mathbf{h}^{(t)} \right) \mathbf{d}^{(t)} (\mathbf{d}^{(t)})^\top, \end{aligned}$$

trong đó $t = 0, 1, 2, \dots$ và

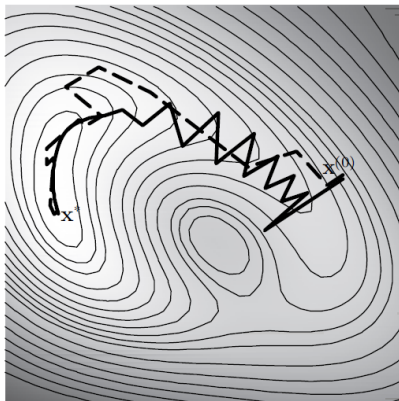
$$\mathbf{d}^{(t)} = \frac{\mathbf{u}^{(t)}}{(\mathbf{h}^{(t)})^\top \mathbf{u}^{(t)}} - \frac{\mathbf{M}^{(t)} \mathbf{h}^{(t)}}{(\mathbf{h}^{(t)})^\top \mathbf{M}^{(t)} \mathbf{h}^{(t)}}.$$

Cách cập nhật này được gọi cập nhật lớp Broyden (**the Broyden class**).

Thông dụng trong lớp này là **cập nhật BFGS** (Broyden - Fletcher - Goldfarb - Shanno), với cách chọn $\delta^{(t)} = 0$.

Việc kiểm tra điều kiện $\left| (\mathbf{v}^{(t)})^\top \mathbf{v}^{(t)} \right| > \epsilon$, với $\epsilon > 0$, là cần thiết. Nếu không thỏa thì $\mathbf{M}^{(t+1)}$ sẽ được đặt là $\mathbf{M}^{(t)}$.

Các phương pháp quasi-Newton



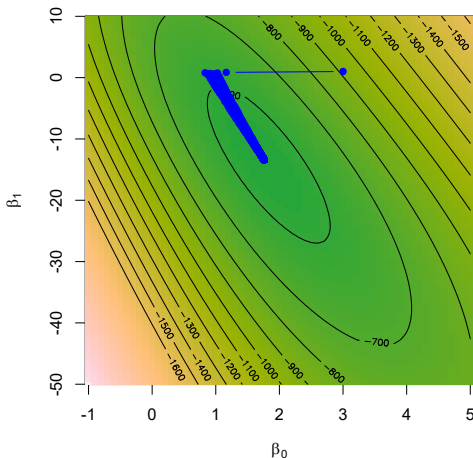
Đường nét đứt màu đen miêu tả đường di chuyển của $x^{(t+1)}$ với cập nhật BFGS.

Thuật toán quasi-Newton với cập nhật BFGS hội tụ thành công, khá nhanh và hiệu quả.

Ví dụ: Human Face Recognition

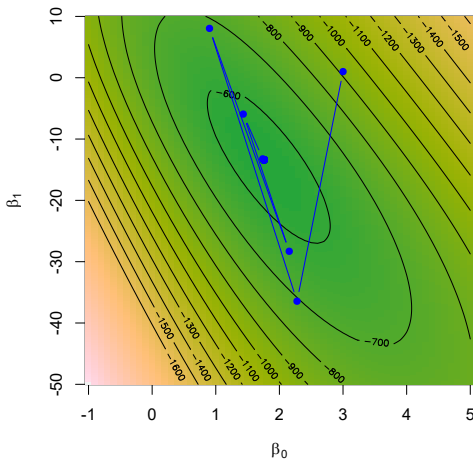
Ta ước lượng lại mô hình đánh giá sự ảnh hưởng của yếu tố eyediff tới xác suất nhận diện đúng $\Pr(\text{match} = 1 | \text{eyediff})$.

Áp dụng steepest ascent, với $\mathbf{M}^{(t)} = -\mathbf{I}_2$, $\beta^{(0)} = (\beta_0^{(0)}, \beta_1^{(0)}) = (3, 1)$.



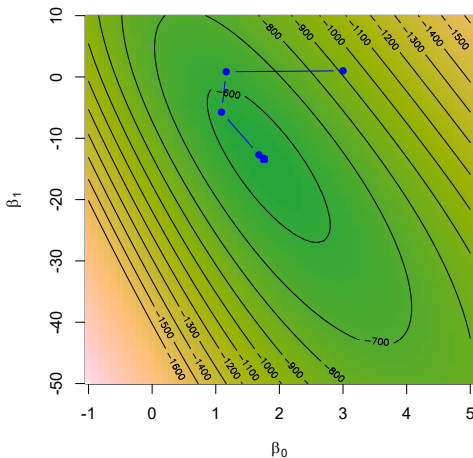
Ví dụ: Human Face Recognition

Áp dụng steepest ascent, với $-\mathbf{M}^{(t)} = \mathcal{I}(\beta^{(t)})$, $\beta^{(0)} = (\beta_0^{(0)}, \beta_1^{(0)}) = (3, 1)$.



Ví dụ: Human Face Recognition

Áp dụng quasi-Newton, cập nhật BFGS, $\mathbf{M}^{(0)} = -\mathbf{I}_2$,
 $\beta^{(0)} = (\beta_0^{(0)}, \beta_1^{(0)}) = (3, 1)$.



Phương pháp Gradient Descent

Nhắc lại, các phương pháp giải lặp Newton hoặc Newton-like, xác định nghiệm lặp theo công thức:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \mathbf{H}_f^{-1}(\mathbf{x}^{(t)}) \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}^{(t)}},$$

hoặc

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \mathbf{M}^{-1}(\mathbf{x}^{(t)}) \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}^{(t)}}.$$

⇒ đều phải xác định ma trận đạo hàm cấp hai dù là chính xác hoặc xấp xỉ.

Phương pháp Gradient Descent

Phương pháp Gradient Descent (GD) đề xuất xác định nghiệm lặp bởi

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha_t \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}^{(t)}},$$

trong đó α_t là bước nhảy (step size) trong từng lần lặp.

Ưu điểm:

- không phải tính ma trận đạo hàm cấp hai
- không phải lưu trữ thông tin $\mathbf{M}^{-1}(\mathbf{x}^{(t)})$ tại bước t ;
- chỉ phải tính vector đạo hàm (gradient).

⇒ áp dụng tốt trong tối ưu nhiều tham số.

Nhược điểm:

- tốc độ hội tụ chậm $O(1/t)$, với t là số bước lặp;
- phải chọn bước nhảy α_t cẩn trọng.

Phương pháp Gradient Descent

Có các phương án tiếp cận cho việc lựa chọn bước nhảy α_t :

- Cố định $\alpha_t = \alpha$, và lựa chọn α trong tập tiềm năng.
- Xác định α_t giảm dần theo lần lặp.
- Tìm kiếm tuyến tính $\Rightarrow \alpha_t$ tương thích.
- Tìm kiếm tương thích với tốc độ học (Adaptive Learning Rate): AdaGrad, RMSProp, Adam.

Phương pháp Gradient Descent - Cố định $\alpha_t = \alpha$

Nếu ta cố định $\alpha_t = \alpha$ với mọi $t = 1, 2, \dots$, thì công nghiệm lặp sẽ là

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}^{(t)}}.$$

Thông thường, tập bước nhảy tiềm năng:

$$\{0.1, 0.01, 0.001, \dots, \}$$

Ưu điểm: dễ áp dụng

Nhược điểm:

- nghiệm không hội tụ nếu α lớn;
- tốc độ hội tụ chậm nếu α nhỏ.

Phương pháp Gradient Descent - Xác định α_t giảm dần

Thông thường, ta có hai cách chọn α_t giảm dần theo t :

- $\alpha_t = \frac{\alpha_0}{t}$ hoặc
- $\alpha_t = \frac{\alpha_0}{\sqrt{t}},$

trong đó α_0 là bước nhảy ban đầu, thường sẽ chọn lớn.

Ưu điểm:

- dễ áp dụng
- nghiệm đảm bảo hội tụ.

Nhược điểm: phụ thuộc vào α_0

Phương pháp Gradient Descent - Tìm kiếm tuyến tính

Trong mỗi bước lặp t , tìm $\alpha_t \leq 1$ lớn nhất, sao cho:

$$f\left(\mathbf{x}^{(t)} - \alpha_t \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}^{(t)}}\right) \leq f(\mathbf{x}^{(t)}) - \beta \alpha_t \left\| \frac{\partial f(\mathbf{x}^{(t)})}{\partial \mathbf{x}^{(t)}} \right\|^2$$

với $0 < \beta \leq 0.5$.

Điều kiện này được là điều kiện Armijo (Armijo condition).

Trong trường hợp điều kiện này không thỏa, ta thay thế (giảm α_t)

$$\alpha_t = \gamma \alpha_t,$$

với $\gamma \in (0, 1)$ và thực hiện tìm kiếm lại sao cho điều kiện Armijo được thỏa.

Ưu điểm:

- tự động hiệu chỉnh,
- ổn định.

Nhược điểm: yêu cầu trúc lặp $\Rightarrow$ chi phí tính toán lớn, nếu α_0 chọn không đủ tốt.